Informe de práctica: sistema de ventas y sistema académico en Python con MVC

Oct 14, 2026 · @asdasdasd

Introducción

Esta práctica construye dos sistemas de consola en Python con arquitectura MVC (Modelo, Vista, Controlador) que guardan sus datos en archivos JSON: un sistema de ventas y un sistema académico. Corresponde a la práctica Nº 2 de Estructura de Datos, carrera de Software de la UNEMI, a cargo del docente Daniel Alexander Vera Paredes.

El objetivo de la guía es construir un sistema académico con código organizado en carpetas, datos guardados en JSON, colecciones y herencia. Para lograrlo se estudió primero el sistema de ventas que trae la guía y después se replicó su arquitectura con estudiantes, materias y matrículas.

El trabajo cubre cuatro aspectos:

- Las cinco operaciones sobre los datos: crear, leer, buscar, actualizar y eliminar.
- Herencia (`ModeloBase` → `Persona` → `Cliente` o `Estudiante`) y polimorfismo mediante el método `etiqueta()` .
- Las cuatro colecciones de Python (`list` , `tuple` , `set` y `dict`), cada una con un uso concreto.
- Pruebas automáticas con distintos tipos de datos y una ejecución completa del menú de cada sistema.

Desarrollo

Los dos sistemas comparten la misma estructura de archivos. El sistema de ventas se desarrolló primero y el académico se construyó sobre él. Cada proceso de la guía se resolvió de la siguiente manera.

Proceso 1: estructura y organización

Se creó la carpeta del proyecto con las subcarpetas `shared/` y `data/` y los archivos `main.py` , `views.py` y `models.py` . El archivo `shared/__init__.py` queda vacío: le indica a Python que `shared` es un paquete y permite importar con `from shared.json_manager import GestorJSON` .

MVC separa el código en tres capas. `models.py` define qué es un dato (Modelo). `views.py` contiene las reglas y el CRUD (Controlador; se llama así por su parecido con Django). `main.py` muestra menús y pide datos (Vista).

Proceso 2: código reutilizable (`shared/`)

- `herramientas.py` : funciones de consola (`imprimir_exito` , `imprimir_error` , `imprimir_titulo` , `confirmar`) y `es_email_valido` . Son funciones y no clases porque no guardan datos.
- `json_manager.py` : la clase `GestorJSON` , la única parte del sistema que toca el disco. `leer()` siempre devuelve una lista, vacía si el archivo no existe o está dañado. `guardar()` escribe en UTF-8 para conservar tildes y la ñ.

Los controladores usan estos módulos y la vista nunca abre un archivo.

Proceso 3: modelos de datos

`ModeloBase` guarda el `id` y define `a_diccionario()` , `etiqueta()` y `__str__()` . También tiene el único método de clase del proyecto, `desde_diccionario()` , que usa `cls(...)` y por eso sirve a todos sus hijos sin reescribirse. `Persona` hereda de ella y agrega nombre, apellido y email.

- **Ventas:** `Cliente(Persona)` agrega teléfono y ciudad. `Producto` concentra lo estático (`STOCK_MINIMO` , `IVA` , `con_iva()` , `es_precio_valido()`). `Venta` calcula su total con un método y guarda el precio del momento como dato histórico.
- **Académico:** `Estudiante(Persona)` agrega carnet y carrera. `Materia` concentra lo estático (`CUPOS_MAXIMOS = 30` , `NOTA_APROBACION = 14` , `es_nota_valida()`). `Matricula` calcula su estado (Aprobado o Reprobado) y no lo guarda.

Proceso 4: lógica CRUD (controladores)

`CrudBase` es una clase abstracta que solo declara los siete métodos de todo controlador: `leer` , `obtener` , `buscar` , `siguiente` , `crear` , `actualizar` y `eliminar` . Cada controlador los implementa con un `for` y un `if` , sin `print()` ni `input()` , y devuelve la tupla `(exito, mensaje)` . Las validaciones ocurren antes de tocar los datos, de modo que un error no deja nada a medio cambiar.

`VentaController` y `MatriculaController` reciben los mismos controladores que usa `main.py` . Así, el stock o el cupo que descuenta una operación ya aparece actualizado en la lista de productos o materias. Anular una venta o retirar una matrícula devuelve el stock o el cupo.

Proceso 5: interfaz de usuario

`mostrar_lista()` y `mostrar_detalle()` son una sola función para los tres modelos de cada sistema, porque cada modelo sabe mostrarse con su `etiqueta()`. Los menús son diccionarios `tecla → (texto, función)` y un único bucle, `ejecutar_menu()`, atiende a todos los submenús.

Proceso 6: integración (`main.py`)

Al arrancar se crea un solo controlador de cada tipo y todas las funciones de pantalla usan esos mismos objetos. `main()` muestra el menú principal (clientes o estudiantes, productos o materias, ventas o matrículas, resumen y salir) y se ejecuta con `python main.py`.

La guía de práctica en PDF nombra una clase `SistemaAcademico`. En este trabajo se siguió la guía HTML, que mantiene `main.py` como funciones y un bucle principal con el mismo efecto.

Colecciones usadas

Colección	Dónde se usa	Por qué se eligió
`list`	`self.lista` de cada controlador	Crece con `append()` y se recorre en orden
`tuple`	Retorno `(exito, mensaje)`, `CAMPOS_BUSCABLES`	Datos fijos que no deben cambiar
`set`	Emails, carnets y códigos únicos; `ciudades()` y `carreras()`; doble matrícula como conjunto de tuplas	Preguntar `in` es instantáneo y no admite repetidos
`dict`	Cada registro del JSON, menús, índices `{id: objeto}`, `ventas_por_cliente()`	Acceso directo por clave

Resultados

Los dos sistemas aprobaron todas sus pruebas automáticas y completaron una sesión simulada del menú de principio a fin. Las pruebas usan una carpeta temporal, por lo que no modifican los archivos reales de `data/`.

Sistema	Pruebas	Correctas	Fallidas
Ventas	104	104	0

Sistema	Pruebas	Correctas	Fallidas
Académico	193	193	0

Tipos de datos probados

Categoría	Datos probados	Resultado
Válidos	Nombres en minúsculas, emails en mayúsculas, precios como texto (`"25.5"`) o número	Se normalizan: Title Case, minúsculas o mayúsculas según el campo
Formato incorrecto	Emails sin `@` o sin dominio, teléfono `09AB` , precio `abc` , `0` , `-5`	Rechazados con un mensaje claro
Límites	Nota 14 y 13.99, créditos 1 y 10, cupos 30 y 31, nota 0 y 20.1	Los límites exactos se aceptan y lo que los excede se rechaza
Tipo incorrecto	`None` , listas, decimales donde va un entero, `nan` e `inf` como nota	Rechazados sin romper el programa
Reglas del negocio	Stock o cupo insuficiente, email, carnet o código repetidos, doble matrícula	Rechazados y los datos no cambian
Persistencia	Reiniciar el programa, JSON corrupto, archivo vacío	Los datos se recuperan; un archivo dañado se lee como lista vacía
Caracteres especiales	Tildes, ñ, texto con comillas, nombres de 600 caracteres	Se guardan y se leen sin pérdida

Sesión simulada del menú

Se ejecutó `main.py` con las entradas de un usuario real, siguiendo el orden de la guía.

Ventas

- Cliente `ana / pérez / ANA@X.com` se guardó como Ana Pérez con el email en minúsculas.
- El mismo email, un teléfono `09AB` y un precio `abc` fueron rechazados con su mensaje.
- El mouse (stock 3) apareció con la alerta de stock bajo, porque `STOCK_MINIMO` es 5.
- La venta de 2 teclados bajó el stock de 6 a 4. Intentar vender 99 mouse devolvió "Stock insuficiente de Mouse: hay 3, piden 99".

- El comprobante mostró subtotal $51.00 y total con IVA $58.65. Al anular la venta, el stock volvió a 6.

Académico

- Un segundo estudiante con el carnet `EST2026001` fue rechazado; el carnet se guarda en mayúsculas.
- La materia con 11 créditos y la que repetía el código `MAT101` fueron rechazadas.
- Matricular dos veces al mismo estudiante en la misma materia y período fue rechazado. Una materia con 0 cupos respondió "No quedan cupos en Física".
- La nota `16,5` (con coma) se registró como 16.5 y la matrícula quedó Aprobada. La nota 25 fue rechazada.
- El certificado mostró estudiante, carnet, materia, créditos, nota y estado. Al retirar una matrícula, el cupo de Física pasó de 0 a 1.

Error encontrado y corregido

Las pruebas del sistema académico detectaron 5 fallas con una misma causa: un valor `None` en un campo de texto se convertía en el texto `"None"` y se aceptaba como dato válido. Se corrigió con la función `como_texto()` en `views.py`, que convierte `None` en texto vacío antes de validar. Con ese cambio las 193 pruebas pasaron.

Conclusiones y recomendaciones

Conclusiones

- **MVC permitió probar la lógica sin el menú.** Como los controladores no usan `print()` ni `input()`, las pruebas automáticas los llaman directamente. Cambiar la consola por otra interfaz solo exigiría reescribir `main.py`.
- **La herencia redujo el código repetido.** `desde_diccionario()` se escribió una vez en `ModeloBase` y la usan los seis modelos. `Estudiante` reutilizó `Persona` con unas pocas líneas nuevas.
- **El polimorfismo evitó `if` por tipo.** `mostrar_lista()` funciona con clientes, productos, ventas, estudiantes, materias y matrículas porque cada modelo define su `etiqueta()`.
- **Cada colección tuvo una razón de ser.** El `set` hizo instantánea la detección de emails, carnets y códigos repetidos. El `dict` dio acceso directo por id en el certificado. La `tuple` evitó cambios en datos fijos.
- **Validar antes de modificar mantuvo los datos consistentes.** Ante un error, el stock, el cupo y los archivos JSON quedaron intactos, como comprobaron las pruebas.

- **Probar con datos distintos encontró un error que el uso normal no mostraba.** Un `None` se guardaba como el texto `"None"` y solo apareció al probar tipos incorrectos.

Recomendaciones

- Para buscar por email entre muchos registros, conviene un diccionario `{email: objeto}` en lugar de recorrer toda la lista, que revisa un elemento tras otro.
- Al agregar un modelo nuevo, añadir también sus pruebas; se puede migrar `pruebas.py` a `unittest` o `pytest`.
- Impedir eliminar un estudiante o una materia que tenga matrículas. Hoy el sistema lo permite y el certificado muestra "eliminado".
- Revisar siempre que los nombres de `a_diccionario()` coincidan con los parámetros del `__init__`, porque `desde_diccionario()` los usa para reconstruir el objeto.

Anexos

Anexo 1. Estructura de carpetas

Los dos proyectos tienen la misma estructura. En `proyecto_ventas` los archivos de datos son `clientes.json`, `productos.json` y `ventas.json`; se crean solos al guardar el primer registro.

```
proyecto_academico/
├── main.py              ← VISTA: menús, input(), print()
├── views.py             ← CONTROLADOR: CRUD de las 3 entidades
├── models.py            ← MODELO: ModeloBase → Persona → Estudiante ·
Materia · Matricula
├── pruebas.py           ← pruebas automáticas
├── shared/
|   ├── __init__.py
|   ├── herramientas.py
|   └── json_manager.py
└── data/
    ├── estudiantes.json
    ├── materias.json
    └── matriculas.json
```

Anexo 2. Resumen de las pruebas automáticas

```
================================================
Pruebas: 104   Correctas: 104   Fallidas: 0     (proyecto_ventas)

================================================
Pruebas: 193   Correctas: 193   Fallidas: 0     (proyecto_academico)
```

Anexo 3. Comprobante de venta (salida real)

```
COMPROBANTE #1   fecha: 2026-10-14
Cliente : Ana Pérez
Producto: Teclado
Cantidad: 2 x $25.50
----------------------------------------
SUBTOTAL: $51.00
TOTAL con IVA: $58.65
```

Anexo 4. Certificado de matrícula (salida real)

```
CERTIFICADO #1   periodo: 2026-1
Estudiante: Ana Pérez   (EST2026001)
Materia   : Cálculo I   (MAT101) · 4 créditos
----------------------------------------
Nota      : 16.5
Estado    : Aprobado
```

Anexo 5. Resumen del sistema académico (salida real)

```
  Estudiantes     : 2
  Carreras        : Redes, Software
  Materias        : 2
  Materias s/cupo : ninguna
  Matrículas      : 1
  Aprobadas       : 1
```

Anexo 6. Archivo `data/matriculas.json` generado

El estado ("Aprobado") no se guarda: se calcula con `estado()` cada vez que se muestra.

```
[
  {
    "id": 1,
    "estudiante_id": 1,
    "materia_id": 1,
    "periodo": "2026-1",
    "nota": 16.5,
    "fecha": "2026-10-14"
  }
]
```

Anexo 7. Código clave

Método de clase heredado por todos los modelos (`models.py`):

```
@classmethod
def desde_diccionario(cls, datos):
    datos = dict(datos)                # copia, para no tocar el original
    id_registro = datos.pop("id")      # saco el id
    return cls(id_registro, **datos)   # cls = la clase real que llamó
```

Validación de doble matrícula con un conjunto de tuplas (`views.py`):

```
existentes = {(m.estudiante_id, m.materia_id, m.periodo) for m in self.lista}
if (estudiante_id, materia_id, periodo) in existentes:
    return "El estudiante ya está matriculado en esa materia en ese período"
```

Corrección del error con `None` (`views.py`):

```
def como_texto(valor):
    return "" if valor is None else str(valor).strip()
```

Referencias bibliográficas

- Bell, D. (2011). *C# para estudiantes*. México D.F.: Pearson Educación.
- Weiss, M. A. (s. f.). *Algoritmos y estructura de datos*. México D.F.: Pearson Educación.
- Universidad Estatal de Milagro. (2026). *Guía de práctica G002-S07-SOFTWARE2019-PRESENCIAL-ESTRUCTURA DE DATOS-A1*, práctica Nº 2.
- *Guía 2: CRUD en Python con MVC y colecciones* (documento HTML de la asignatura).